

loadgen - Start Here

MongoDB Load Generator + Validation Harness. Generate known, characterized MongoDB load and diff its effects against the target's FTDC output. Works against self-managed replica sets and MongoDB Atlas, driven entirely from a tabbed web UI. This guide: get it, run it, customize, schedule, and the caveats.

Repo: <https://github.com/looking4manish/mdb-loadtest> (MIT). Requires Python 3.10+.

1) Get the repo

```
git clone git@github.com:looking4manish/mdb-loadtest.git
cd mdb-loadtest
```

2) Deploy + run (one command)

The deploy script creates a virtualenv, installs dependencies, verifies them, and (with the run flag) starts the web app.

macOS / Linux:	<code>./deploy.sh --run</code>	# PORT=9000 <code>./deploy.sh --run</code> to change port
Windows (PS):	<code>.\deploy.ps1 -Run</code>	# add -Port 8077 if 8000 is reserved

Then open the URL it prints, e.g. `http://127.0.0.1:8000/` (or your chosen port).

Manual equivalent: `python -m venv venv ; venv/bin/python -m pip install -r requirements.txt ; venv/bin/python -m uvicorn app:app --host 127.0.0.1 --port 8000` (Windows: `venv\Scripts\python`).

3) Drive a test from the UI (3 tabs; use Next / Previous)

Live Log and the Danger Zone (teardown) sit below the tabs, always visible. The top-right icon button cycles the neon glow color (persists).

- Tab 1 - Connection: paste the full connection URI, then 'Test Connection + Permission Check'. It reports server version, topology, whether it is PRIMARY, edition, runs a 6-capability permission probe, and a clock-skew check. The password is masked and never logged.
- Tab 2 - Output Details: set the editable database name, validate the output folder, review the Expected Collections panel, set seed doc counts, and click 'Seed now' (idempotent).
- Tab 3 - Workloads & Run: tick workloads + tune their parameters, then 'Start Run' (set a duration) OR schedule a recurring window. Output goes to `runs/run_<timestamp>_<id>/` with a manifest.json (incl. before/after opcounter deltas proving the load hit the server) + run.log.

4) Customize the values

- Connection URI - the single source of truth for the target. Examples:

Self-managed / replica set:	<code>mongodb://USER:PASSWORD@HOST:27017/?authSource=admin</code>
MongoDB Atlas (SRV, TLS):	<code>mongodb+srv://USER:PASSWORD@cluster.xxxxx.mongodb.net/</code>

- Auth source / Database name: override fields on the Connection + Output tabs (default db 'loadtest').
- Output folder: where manifests + logs are written (default `./runs`).
- Seed sizes: large_dataset / agg_dataset / hot_docs counts (start small, e.g. 50000, scale up).
- Per-workload params: threads, target ops/sec, batch size, doc KB, hot-doc count, pipeline complexity, blend weights/seed - all editable inline under each ticked workload.
- Run: duration (seconds), optional run seed, 'Auto-seed before run', and an override to proceed despite clock skew (not recommended).

5) Schedule a recurring window (Tab 3)

Persistence dropdown:

- In-app (APScheduler + SQLite): survives an app RESTART, but only fires WHILE the app runs.
- Permanent (OS scheduler): registers a real OS task that runs with the app CLOSED and survives reboot. The UI picks the backend by platform and auto-seeds before each run.

Set start time, end, days, per-day duration, base seed, jitter (random delay), then Schedule. 'Refresh list' shows both in-app jobs and permanent OS tasks; 'Remove permanent' deletes the OS task.

Where the permanent task is stored (per OS)

- WINDOWS: Task Scheduler task 'loadgen-daily'. Inspect: `schtasks /query /TN loadgen-daily` (or `taskschd.msc`). Config + URI live in `win_tasks\loadgen-daily.cmd` (icaccls-locked). Jitter = the task's RandomDelay.
- macOS: `launchd` LaunchAgent `~/Library/LaunchAgents/com.loadgen.daily.plist` (`launchctl bootstrap`). Inspect: `launchctl print gui/$(id -u)/com.loadgen.daily`. URI lives in `mac_tasks/loadgen-daily.sh` (`chmod 600`). Jitter = random sleep in `run_window.py`. Self-removes after the end date. Reference plist in `launchd/`.
- LINUX: UI button not wired - install the `journald`-logged `systemd` timer in `systemd/` (see README).

On Windows and macOS the task runs WHILE YOU ARE LOGGED ON and auto-starts after reboot/login. Running while fully logged off needs elevation (a Windows S4U task / a root LaunchDaemon).

Teardown (when done)

Drops the load-test database and removes all scheduled jobs. Use the red UI button, or the CLI:

```
venv/bin/python teardown.py "mongodb://USER:PASSWORD@HOST:27017/?authSource=admin" loadtest
```

Caveats

- Port 8000 is reserved on some Windows hosts (winerror 10013) - pick another port (e.g. 8077).
- Permanent schedule runs only while logged on (see above); fully logged-off needs admin/root.
- The macOS `launchd` path is implemented but NOT yet verified on a real Mac - confirm with the `launchctl print` command before relying on it.
- The in-app scheduler stores the connection URI (incl. password) in cleartext in `loadgen_jobs.sqlite`. That file is git-ignored (never pushed), but it is on disk - prefer the permanent mode (locked wrapper) or remove in-app jobs after use.
- Clock skew > 2s between the driver host and the target aborts a run (it would corrupt FTDC correlation) - keep NTP in sync. Connection errors are mapped to a named cause (DNS/SRV, IP-not-allowlisted/firewall, auth failed, TLS).
- Run the driver from a host that is NOT a replica-set member, so the load driver stays out of the metrics being validated.

Verified: connection, permission check, seed, a mixed run (with opcounter deltas), manifest, and in-app + Windows permanent scheduling all pass against a MongoDB 8.2 Enterprise replica set. macOS launchd is implemented, pending Mac verification. Full details: README.md and EXPECTED_SIGNALS.md.